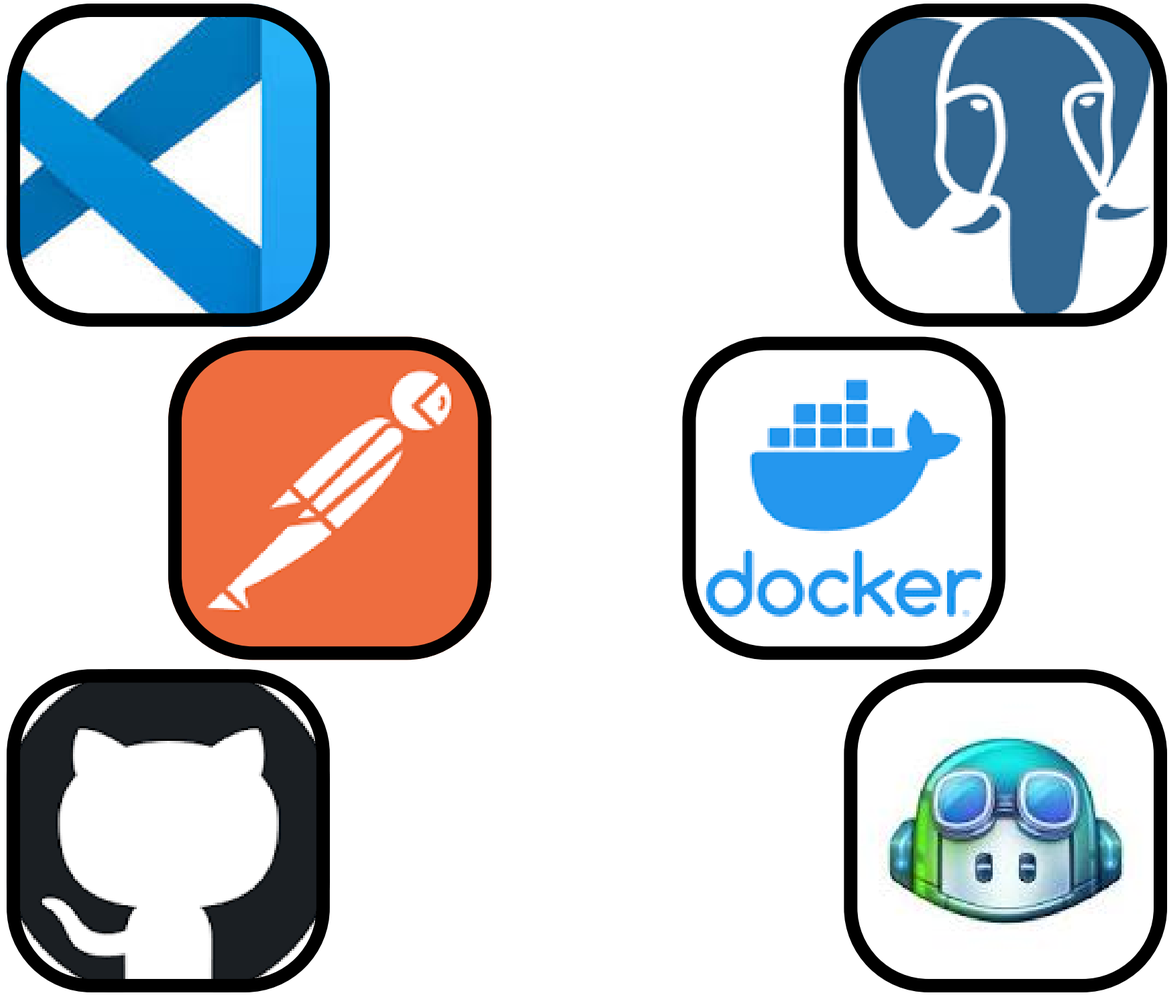
Par Xabi Martinez, le 20/11/2025

Organisation

Projet individuel



Contexte et objectifs du projet

- Projet : backend en Rust
- Objectifs : modéliser, sécuriser, exposer, tester, déployer
- Architecture API REST + PostgreSQL

Contraintes du projet

- Reconversion + activité professionnelle
- Gestion stricte du temps
- Rust : courbe d’apprentissage élevée
- Approche MVP puis itérations
- Documentation rigoureuse

Architecture globale

- API REST Axum
- PostgreSQL + SQLx
- Dual JWT
- Utilisateur <=> API <=> Base de données
- Build Docker multi-stage

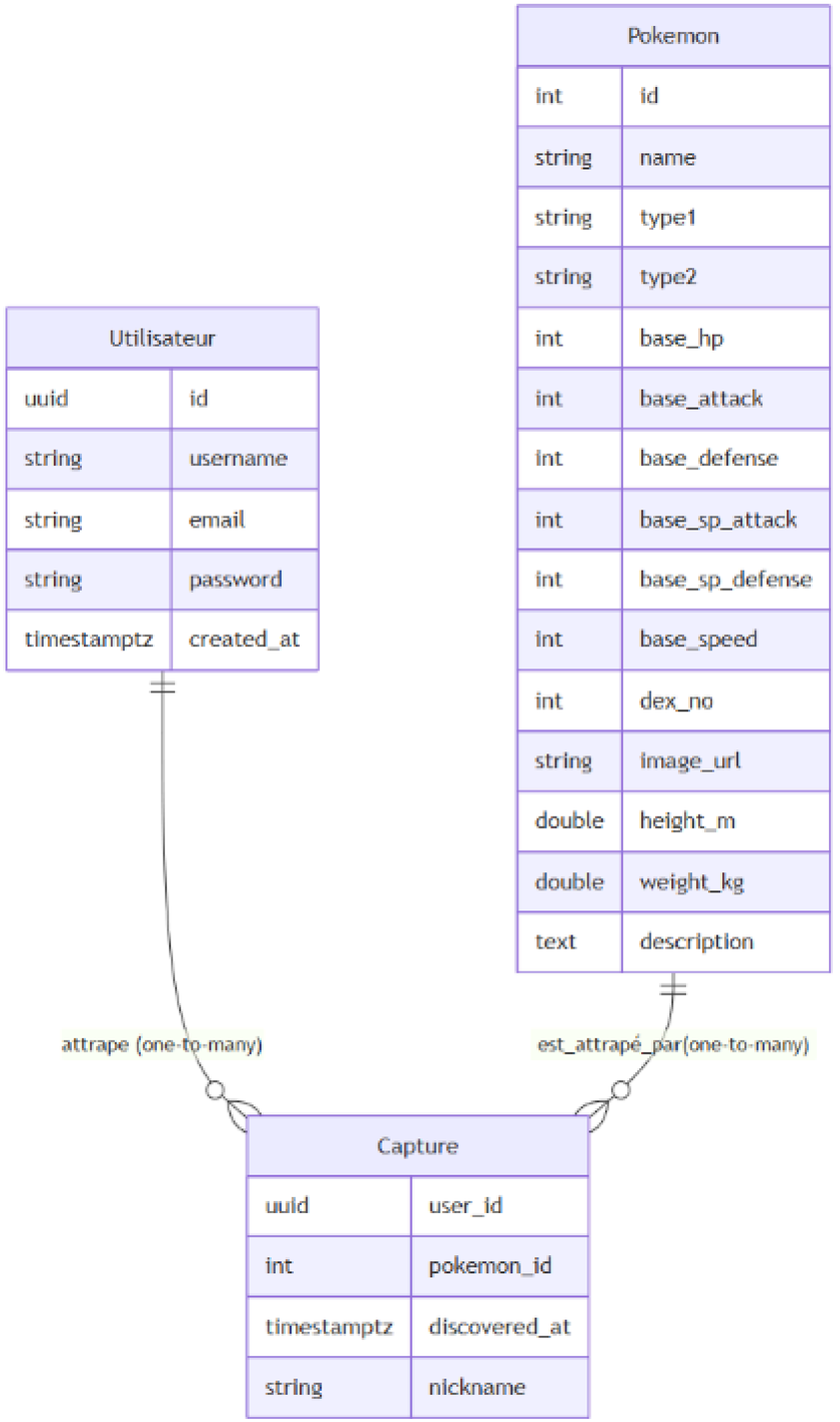
User stories

- peut créer un compte
- peut supprimer son compte
- peut se connecter
- peut se déconnecter
- peut voir son profil
- peut modifier son mot de passe
- peut lister tous les Pokémon
- peut capturer un Pokémon
- peut voir le détail d'un Pokémon
- peut rechercher un Pokémon
- par nom ou numéro

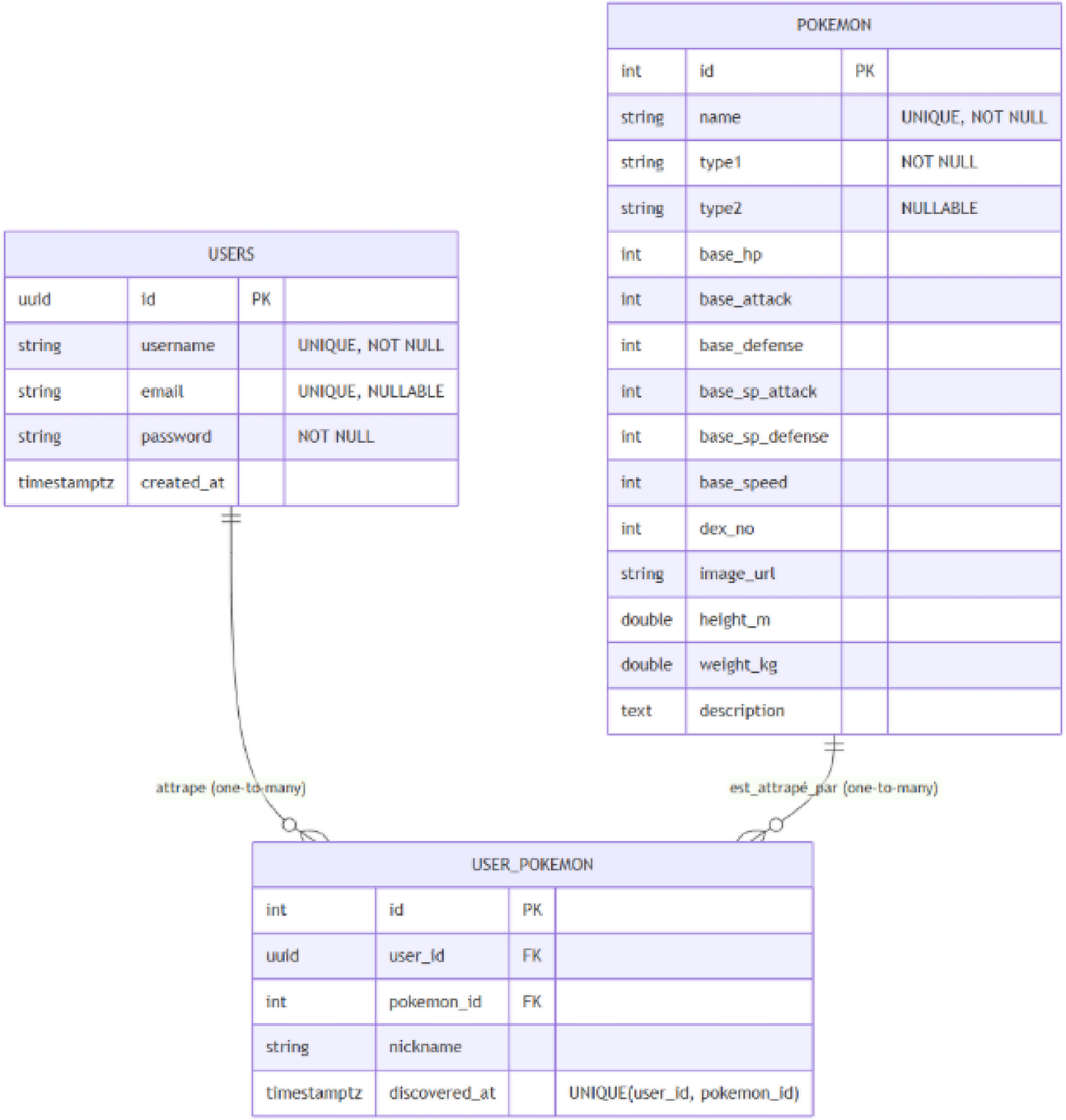
Conception Merise et contraintes

- MCD : Users, Pokémon, User_Pokemon
- Relation N<=>N capturée
- Attribut dérivé caught
- Contraintes UNIQUE + cascade
- Modèle stable & normalisé

MCD


Pokemon
int id
string name
string type1
string type2
int base_hp
int base_attack
int base_defense
int base_sp_attack
int base_sp_defense
int base_speed
int dex_no
string image_url
double height_m
double weight_kg
text description
Utilisateur
uuid id
string username
string email
string password
timestamptz created_at
attrape (one-to-many)
est_attrapé_par(one-to-many)
Capture
uuid user_id
int pokemon_id
timestamptz discovered_at
string nickname

MPD


USERS
uuid id PK
string username UNIQUE, NOT NULL
string email UNIQUE, NULLABLE
string password NOT NULL
timestamptz created_at
POKEMON
int id PK
string name UNIQUE, NOT NULL
string type1 NOT NULL
string type2 NULLABLE
int base_hp
int base_attack
int base_defense
int base_sp_attack
int base_sp_defense
int base_speed
int dex_no
string image_url
double height_m
double weight_kg
text description
attrape (one-to-many)
est_attrapé_par (one-to-many)
USER_POKEMON
int id PK
uuid user_id FK
int pokemon_id FK
string nickname
timestamptz discovered_at UNIQUE(user_id, pokemon_id)

Initialisation DB

```sql
-- ===== TABLE USERS =====
CREATE TABLE IF NOT EXISTS users (
    id UUID PRIMARY KEY DEFAULT gen_random_uuid(),
    username VARCHAR(50) UNIQUE NOT NULL,
    email VARCHAR(100) UNIQUE,
    password TEXT NOT NULL,
    created_at TIMESTAMPTZ NOT NULL DEFAULT NOW()
);

-- ===== TABLE POKEMON =====
CREATE TABLE IF NOT EXISTS pokemon (
    id SERIAL PRIMARY KEY,
    name VARCHAR(100) UNIQUE NOT NULL,
    type1 VARCHAR(20) NOT NULL,
    type2 VARCHAR(20),
    base_hp INTEGER,
    base_attack INTEGER,
    base_defense INTEGER,
    base_sp_attack INTEGER,
    base_sp_defense INTEGER,
    base_speed INTEGER
);

-- ===== TABLE ASSOCIATIVE USER_POKEMON =====
CREATE TABLE IF NOT EXISTS user_pokemon (
    id SERIAL PRIMARY KEY,
    user_id UUID NOT NULL REFERENCES users(id) ON DELETE CASCADE,
    pokemon_id INTEGER NOT NULL REFERENCES pokemon(id),
    nickname VARCHAR(50),
    discovered_at TIMESTAMPTZ NOT NULL DEFAULT NOW(),
    UNIQUE(user_id, pokemon_id)
);

-- ===== INDEX =====
CREATE INDEX IF NOT EXISTS idx_user_pokemon_user_id ON user_pokemon(user_id);
CREATE INDEX IF NOT EXISTS idx_user_pokemon_pokemon_id ON user_pokemon(pokemon_id);
```

Migration DB

```sql
ALTER TABLE pokemon
    ADD COLUMN IF NOT EXISTS dex_no INTEGER,
    ADD COLUMN IF NOT EXISTS image_url TEXT;
```

```sql
ALTER TABLE pokemon
    ADD COLUMN IF NOT EXISTS height_m DOUBLE PRECISION,
    ADD COLUMN IF NOT EXISTS weight_kg DOUBLE PRECISION,
    ADD COLUMN IF NOT EXISTS weaknesses TEXT[];
```

```sql
ALTER TABLE pokemon
    DROP COLUMN IF EXISTS weaknesses,
    ADD COLUMN IF NOT EXISTS description TEXT;
```

Sécurité de l’API

- JWT access court / refresh long
- Cookies HttpOnly, SameSite Lax/Strict
- Argon2 pour les mots de passe
- Middlewares Axum
- Mitigation CSRF (cookies + header)

Flux applicatifs

- Login : validation + tokens
- Refresh automatique
- Listing Pokémon
- Capture (idempotente)
- Vérifications & permissions

Extraits de code

catch function

```rust
pub async fn catch(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
    Json(payload): Json<CatchByNamePayload>,
) -> ApiResult<(axum::http::StatusCode, String)> {
    let pokemon_id = sqlx::query_scalar::<_, i32>(r#"SELECT id FROM pokemon WHERE name = $1"#)
        .bind(&payload.name)
        .fetch_optional(&pool)
        .await
        .map_err(to_500)?
        .ok_or_else(|| not_found("Pokémon introuvable."))?;

    let _ = sqlx::query(
        r#"
        INSERT INTO user_pokemon (user_id, pokemon_id, nickname)
        VALUES ($1, $2, $3)
        ON CONFLICT (user_id, pokemon_id) DO NOTHING
        "#,
    )
    .bind(user_id)
    .bind(pokemon_id)
    .bind(payload.nickname)
    .execute(&pool)
    .await
    .map_err(to_500)?;

    created("Pokémon marqué comme capturé.")
}
```

list_all function

```rust
pub async fn list_all(
    CurrentUser(user_id): CurrentUser,
    State(pool): State<PgPool>,
) -> ApiResult<Json<Vec<PokemonWithCaught>>> {
    let rows = sqlx::query_as::<_, PokemonWithCaught>(
        r#"
        SELECT
            p.id          AS id,
            p.name        AS name,
            p.type1       AS type1,
            p.type2       AS type2,
            p.dex_no      AS dex_no,
            p.image_url   AS image_url,
            EXISTS (
                SELECT 1 FROM user_pokemon up
                WHERE up.user_id = $1 AND up.pokemon_id = p.id
            )             AS caught
        FROM pokemon p
        ORDER BY p.id
        "#,
    )
    .bind(user_id)
    .fetch_all(&pool)
    .await
    .map_err(to_500)?;

    Ok(Json(rows))
}
```

Performance et optimisations

- Rust + Tokio
- Requêtes compilées SQLx
- Démarrage : retry DB
- Idempotence et LIMIT
- Calcul dynamique caught

Qualité et tests

- Tests d’intégration (reqwest)
- nextest : exécution rapide
- Linter clippy
- Séparation clean du code
- Documentation claire

Tests

```
    Nextest run ID 30847ad9-3b4b-483a-a030-82761db22e59 with nextest profile: default
       Starting 17 tests across 8 binaries
            PASS [   0.709s] pokedex_rncp_backend::health_test get_root_retourne_bienvenue
            PASS [   0.710s] pokedex_rncp_backend::db_test la_base_de_donnees_repond
            PASS [   0.718s] pokedex_rncp_backend::auth logout_definit_cookies_expirants
            PASS [   0.712s] pokedex_rncp_backend::health_test path_inconnu_retourne_404
            PASS [   0.727s] pokedex_rncp_backend::pokemon list_requiert_auth
            PASS [   0.744s] pokedex_rncp_backend::auth me_requiert_authentification
            PASS [   0.741s] pokedex_rncp_backend::auth tokens_invalides_donnent_401
            PASS [   1.503s] pokedex_rncp_backend::auth me_ok_avec_bearer
            PASS [   1.542s] pokedex_rncp_backend::auth me_ok_avec_cookie_auth
            PASS [   1.554s] pokedex_rncp_backend::auth refresh_token_ok_avec_cookie_refresh
            PASS [   1.555s] pokedex_rncp_backend::auth refresh_token_requiert_bearer_et_definit_cookie
            PASS [   1.130s] pokedex_rncp_backend::pokemon list_search_catch_get
            PASS [   1.172s] pokedex_rncp_backend::user delete_user_soi_meme
            PASS [   1.604s] pokedex_rncp_backend::user create_user_201_et_conflit_409
            PASS [   1.890s] pokedex_rncp_backend::user update_user_refuse_si_different
            PASS [   2.219s] pokedex_rncp_backend::user update_user_soi_meme_modifie_username_email_et_password
            PASS [   3.182s] pokedex_rncp_backend::auth reset_mot_de_passe_retourne_token_et_confirmation_mise_a_jour
```

Crates

Tokio

A runtime for writing reliable, asynchronous, and slim applications with the Rust programming language. It is:

- **Fast**: Tokio's zero-cost abstractions give you bare-metal performance.
- **Reliable**: Tokio leverages Rust's ownership, type system, and concurrency model to reduce bugs and ensure thread safety.
- **Scalable**: Tokio has a minimal footprint, and handles backpressure and cancellation naturally.

Website | Guides | API Docs | Chat

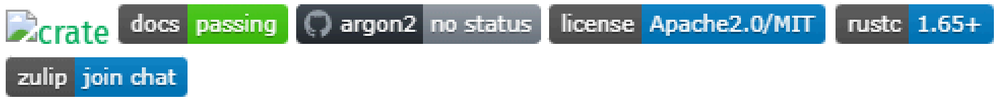
Overview

Tokio is an event-driven, non-blocking I/O platform for writing asynchronous applications with the Rust programming language. At a high level, it provides a few major components:

- A multithreaded, work-stealing based task scheduler.
- A reactor backed by the operating system's event queue (epoll, kqueue, IOCP, etc.).
- Asynchronous TCP and UDP sockets.

These components provide the runtime components necessary for building an asynchronous application.

RustCrypto: Argon2



Pure Rust implementation of the Argon2 password hashing function.

Documentation

About

Argon2 is a memory-hard key derivation function chosen as the winner of the Password Hashing Competition in July 2015.

It implements the following three algorithmic variants:

- **Argon2d**: maximizes resistance to GPU cracking attacks
- **Argon2i**: optimized to resist side-channel attacks
- **Argon2id**: (default) hybrid version combining both Argon2i and Argon2d

Support is provided for embedded (i.e. `no_std`) environments, including ones without `alloc` support.

Déploiement professionnel

- Docker multi-stage Rust
- docker-compose (API + DB)
- Images légères
- Variables d’environnement
- Tunnel HTTPS possible

Compétences RNCP couvertes

- Modélisation Merise & SQL relationnel
- Développement d’API REST sécurisée
- Sécurité : hachage, JWT, cookies
- Tests, conteneurisation, déploiement

Conclusion

- Projet complet & opérationnel
- Rust : fiabilité & rigueur
- Approche professionnelle
- Sécurité & qualité au cœur

Merci pour votre attention !